

1. Process Concept, Concurrency & Race Conditions

Syllabus Coverage: Gateway Classes Slides 3–15 | Process Concepts, Concurrency Principles, Independent vs Cooperating Processes, Race Conditions.

1.1 Process Concept & Memory Organization

Program in Execution: Hard disk par program ek passive byte file hota hai. Jab OS isse RAM mein load karta hai, toh yeh **active process** ban jata hai jiske paas apna context aur resource allocation hota hai.

- **Text Section:** Program instructions (code segment).
- **Data Section:** Global aur static variables.
- **Heap:** Dynamic runtime memory (via `malloc`).
- **Stack:** Local activation records, function parameters aur return addresses.
- **Process Control Block (PCB):** Kernel data structure containing PID, Program Counter, CPU registers, scheduling priority, aur open file descriptors.

1.2 Independent vs Cooperating Processes

Parameter	Independent Process	Cooperating Process
Shared State	Koi shared data ya resource nahi hota.	Shared variables, buffer, files ya hardware resources.
Execution Effect	Kisi doosre process ko affect nahi karta.	Doosre process ke execution ko directly modify kar sakta hai.
Determinism	Deterministic: Same input = Always same result.	Non-deterministic: Outcome depends on execution order.
Risk	Zero synchronization hazard.	Race Condition & Data Inconsistency risk!

1.3 Race Condition Execution Trace

Jab concurrent processes bina synchronization ke shared data par read/write karte hain, toh outcome interleaved execution order par depend karta hai.

```
; Initial Shared Variable: Balance = 1000
; Process P1 (Deposit 500)
1. LOAD  R1, Balance    ; R1 = 1000
2. ADD   R1, 500        ; R1 = 1500
; --- Context Switch to P2 ---
3. LOAD  R2, Balance    ; R2 = 1000 (stale value!)
4. SUB   R2, 200        ; R2 = 800
5. STORE Balance, R2    ; RAM Balance = 800
; --- Context Switch back to P1 ---
6. STORE Balance, R1    ; RAM Balance = 1500! (Withdrawal lost!)
```

Process Concurrency & Race Condition Architecture

Independent vs Cooperating Processes & Data Inconsistency

1. Independent Processes (Deterministic)

- Does NOT share memory, variables or I/O state.
 - Execution sequence does not affect other processes.
 - Deterministic output: Same input = Always same output.
- Example:** Calculator and Music Player running simultaneously.

✓ No Synchronization Required

2. Cooperating Processes (Non-Deterministic)

- Shares state, shared memory, files, or variables.
 - Can affect or be affected by other concurrent processes.
 - Non-deterministic: Order of execution dictates final output.
- Example:** Producer-Consumer, ATM balance concurrent withdraw.

⚠ Prone to Race Condition & Data Inconsistency!

⚠ The Race Condition: Bank Account Balance = 1000

Process P1 (Deposit 500)

R1 = Balance (1000)
R1 = R1 + 500 (1500)
[Preempted before store!]
Later: Balance = 1500

Shared Memory: Balance
Initial Value: 1000

Final Corrupt Value: 800 or 1500!

Process P2 (Withdraw 200)

R2 = Balance (1000)
R2 = R2 - 200 (800)
Balance = 800 (Stored!)
P1 overwrites later!

1.4 Exam & Interview Summary

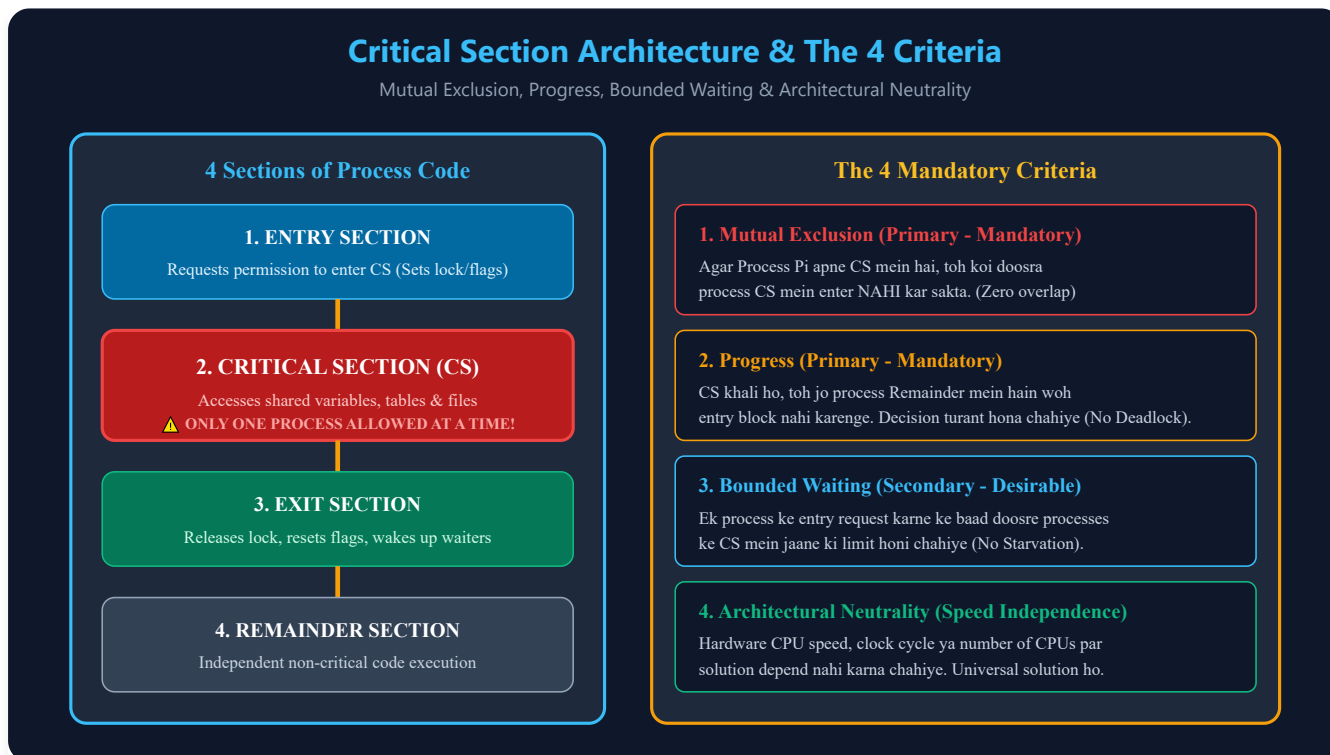
- **Core Rule:** Shared data + Preemption = Race Condition.
- **Solution Required:** Mutual Exclusion around shared code (Critical Section).

2. Critical Section Problem & The 4 Criteria

Syllabus Coverage: Gateway Classes Slides 16–24 | 4 Sections of Process Code, Primary vs Secondary Criteria.

2.1 The 4 Sections of Process Code

- **Entry Section:** Lock acquire karne aur CS mein ghusne ki permission ka code.
- **Critical Section (CS):** Shared data manipulation code (Only ONE process allowed).
- **Exit Section:** Lock release aur waiting processes ko inform karne ka code.
- **Remainder Section:** Baki ka normal independent code execution.



2.2 The 4 Solution Criteria

Criterion	Type	Core Definition	Prevention Goal
Mutual Exclusion	Primary (Mandatory)	Only one process can execute in CS at any given time.	Data Inconsistency
Progress	Primary (Mandatory)	Selection of next process cannot be blocked by remainder processes or postponed indefinitely.	Deadlock
Bounded Waiting	Secondary (Desirable)	Bound on the number of entries granted to others after a request is made.	Starvation
Architectural Neutrality	Secondary (Desirable)	No assumptions regarding relative CPU speeds or hardware clocks.	Portability bugs

2.3 Common Interview Traps

Trap: Kya Bounded Waiting satisfy na hone par solution reject ho jata hai?

Answer: Primary conditions (Mutual Exclusion + Progress) strictly mandatory hain. Agar Bounded Waiting satisfy na ho, toh solution partial mana jata hai (starvation prone), lekin system deadlock me nahi fasata.

3. Peterson's Algorithm (2-Process Synchronization)

Syllabus Coverage: Gateway Classes Slides 25–39 | 2-Process Software Solution, Step-by-Step Code Trace, Mathematical Proofs.

3.1 Core Architecture & Shared Variables

- `boolean flag[2] = {false, false};` — Process intention array (`flag[i] = true` means P_i is interested).
- `int turn;` — Decides tie-breaking priority when both processes contend simultaneously.

Peterson's Algorithm: 2-Process Synchronization

State Trace: `flag[0]`, `flag[1]` & `turn` Variable Mechanics

Process P0 ($i = 0$, other = 1)

```
1. flag[0] = true; // Main interested hoon
2. turn = 1; // Pehle aap bhai (P1)
3. while (flag[1] && turn == 1); // Busy Wait
4. /* CRITICAL SECTION */
5. flag[0] = false; // Done, main bahar hoon
6. /* Remainder Section */
```

Key Logic:

P0 expresses interest, gives priority to P1. P0 enters if P1 is not interested OR `turn` gets updated to 0!

Process P1 ($j = 1$, other = 0)

```
1. flag[1] = true; // Main interested hoon
2. turn = 0; // Pehle aap bhai (P0)
3. while (flag[0] && turn == 0); // Busy Wait
4. /* CRITICAL SECTION */
5. flag[1] = false; // Done, main bahar hoon
6. /* Remainder Section */
```

Key Logic:

P1 expresses interest, gives priority to P0. P1 enters if P0 is not interested OR `turn` gets updated to 1!

Verification of Peterson's Solution against the 3 Criteria

1. Mutual Exclusion: ✓ YES

Dono ek saath enter nahi ho sakte kyunki `turn` ek hi integer hai (either 0 or 1, not both!).

2. Progress: ✓ YES

Agar P1 Remainder me hai toh `flag[1]=false` hoga, P0 bina kisi delay ke enter kar jayega.

3. Bounded Waiting: ✓ YES

Waiting bound $k = 1$. Ek process CS se nikalte hi doosre process ko mauka milta hai (No Starvation).

3.2 Step-by-Step Execution Trace

```
// Process P0 Code
flag[0] = true;
turn = 1;
while (flag[1] && turn == 1); // Busy waiting
/* CRITICAL SECTION */
flag[0] = false;
/* Remainder Section */

// Process P1 Code
flag[1] = true;
turn = 0;
while (flag[0] && turn == 0); // Busy waiting
/* CRITICAL SECTION */
flag[1] = false;
/* Remainder Section */
```

3.3 Criteria Proof Summary

Criterion	Status	Mathematical / Logical Proof
Mutual Exclusion	✓ Satisfied	Variable <code>turn</code> cannot be both 0 and 1 at the same nanosecond. Only one process can evaluate while loop to false.
Progress	✓ Satisfied	If P1 is in Remainder, <code>flag[1] == false</code> , allowing P0 to enter immediately without delay.
Bounded Waiting	✓ Satisfied	Bound $k=1$. P0 exiting sets <code>flag[0]=false</code> ; if re-requesting, sets <code>turn=1</code> , granting P1 immediate entry.

4. Software Solutions: Dekker's Algorithm

Syllabus Coverage: Gateway Classes Slides 108–120 | 1st Correct Software Mutual Exclusion Algorithm, Collision Resolution, Comparison with Peterson.

4.1 Core Code & Logic

```
flag[i] = true;
while (flag[j]) {
    if (turn == j) {
        flag[i] = false;    // Polite back-off to prevent Livelock
        while (turn == j);  // Wait until Pj exits CS
        flag[i] = true;    // Re-assert intention
    }
}
/* CRITICAL SECTION */
turn = j;
flag[i] = false;
```

Dekker's Algorithm: First Software Mutual Exclusion

Contention Back-off Loop & Tie-Breaker Turn Mechanism

Process Pi (Pi & Pj Contention Flow)

```
flag[i] = true; // 1. Declare intent
while (flag[j]) { // 2. Is Pj interested?
    if (turn == j) { // 3. If turn belongs to Pj:
        flag[i] = false; // Pi backs off (yields)
        while (turn == j); // Wait until Pj finishes
        flag[i] = true; // Pi re-asserts intent
    }
}
/* CRITICAL SECTION */
turn = j; // Give next turn to Pj
flag[i] = false; // Release intention
/* Remainder Section */
```

Dekker's vs Peterson's Comparison

1. Complexity & Structure

Dekker requires nested while loops & explicit back-off. Peterson is a single elegant while loop condition.

2. Intent Withdrawal (Yielding)

Dekker temporarily sets `flag[i]=false` to prevent livelock. Peterson never resets flag until exiting CS completely.

3. Historical Context

Dekker (1965) was the 1st correct software algorithm. Peterson (1981) simplified it 16 years later.

Mutual Exclusion: ✓ YES

Dono ek saath CS mein kabhi nahi ghus sakte kyunki turn collision break karta hai.

Progress: ✓ YES

Inactive process dusre ko nahi rokta. Livelock back-off logic se resolve hota hai.

Bounded Waiting: ✓ YES

Exiting process explicitly `turn = j` pass karta hai, preventing indefinite starvation.

Limitation: Busy Waiting

Both while loops consume continuous CPU cycles (Spinlocks).

4.2 Comparison Table: Dekker vs Peterson

Feature	Dekker's Algorithm (1965)	Peterson's Algorithm (1981)
Structure	Nested loops with explicit back-off.	Single concise while condition.
Intention Reset	Temporarily drops flag to false during conflict.	Keeps flag true; modifies <code>turn</code> only.
Criteria Satisfied	Mutual Exclusion, Progress, Bounded Waiting.	Mutual Exclusion, Progress, Bounded Waiting.
Instruction Count	Higher (more branches and checks).	Lower (minimal overhead).

5. Hardware Synchronization: Test-and-Set & Swap

Syllabus Coverage: Gateway Classes Slides 40–59 | Software Lock Flaw, Atomic TSL Primitives, Swap Instructions, Spinlock Evaluation.

5.1 The Software Lock Variable Flaw

Preemption between the *test* (`while(lock==1)`) and *set* (`lock=1`) operations allows multiple processes to enter the Critical Section simultaneously, violating Mutual Exclusion.

5.2 Hardware Atomic Test_and_Set (TSL)

```
// Hardware Atomic Primitive
boolean Test_and_Set(boolean *target) {
    boolean r = *target;
    *target = true;
    return r;
}

// Process Synchronization Code
while (Test_and_Set(&lock)); // Spinlock busy waiting
/* CRITICAL SECTION */
lock = false; // Release lock
```

Hardware Synchronization: Test-and-Set (TSL) & Swap

Atomic Instruction Execution & Spinlock Mechanics

Why Simple Software Lock Fails (Flaw)

```
while (lock == 1); // 1. Check lock
// <--- PREEMPTION OCCURS HERE! --->
lock = 1; // 2. Set lock
/* CRITICAL SECTION */
```

The Flaw:

Agar lock 0 tha aur P1 ne test karke preemption le li, toh P2 bhi lock 0 dekh kar CS me chala jayega! Mutual Exclusion FAILS!

Hardware Atomic Test_and_Set (TSL)

```
boolean Test_and_Set(boolean *target) {
    boolean r = *target; // Read original
    *target = true; // Set to 1
    return r; // Return original
} // <--- EXECUTED IN 1 ATOMIC CLOCK CYCLE!
```

Usage in Code:

```
while (Test_and_Set(&lock)); /* CS */ lock = false;
```

Spinlock Architecture & Criteria Verification

What is a Spinlock?

• A lock where waiting thread spins continuously in while loop.

Good for: Short waiting time.

Bad for: Long wait (wastes CPU).

The Swap Instruction

```
void Swap(boolean *a, *b) {
    temp=*a; *a=*b; *b=temp;
}

key = true;
while(key) Swap(&lock, &key);
```

Criteria Evaluation

• Mutual Exclusion: ✓ YES

• Progress: ✓ YES

• Bounded Waiting: ✗ NO!

Random scheduling may cause a process to starve!

5.3 Criteria Satisfaction Summary

Criterion	Status	Technical Reason
Mutual Exclusion	✓ YES	Atomic memory bus lock prevents concurrent false returns.

Progress	✓ YES	No external or remainder process blocks CS entry.
Bounded Waiting	X NO	Random hardware spin contention cannot guarantee bounded turns (Starvation possible).

6. Semaphores and Mutexes

Syllabus Coverage: Gateway Classes Slides 60–75 | Counting vs Binary Semaphores, Block-Wakeup Implementation, Mutex vs Semaphore.

6.1 Semaphore Concept & Operations

An integer variable accessed strictly via atomic `wait()` and `signal()` operations.

- **Binary Semaphore:** Value restricted to 0 or 1. Used for mutual exclusion.
- **Counting Semaphore:** Value 0 to N. Used for tracking pools of finite resources.

Semaphores & Mutexes Architecture

Counting vs Binary Semaphores & Block-Wakeup Queue Implementation

Semaphore Types & Definitions

1. Binary Semaphore (Value: 0 or 1)

- Used for Mutual Exclusion (Locking).
- Similar to Mutex, but lacks thread ownership.

Initial value = 1 (Unlocked)

2. Counting Semaphore (Value: 0 to N)

- Used for resource management (Pool of N units).
- E.g., Database connection pool, 5 printers.

Initial value = N (Available resource count)

Block & Wakeup (No Busy Waiting)

```
typedef struct {
    int value;
    struct process *list; // Waiting PCB queue
} semaphore;
```

```
wait(S) { S.value--;
    if (S.value < 0) { add to S.list; block(); } }

signal(S) { S.value++;
    if (S.value <= 0) { wakeup(P from list); } }
```

[S.value] when negative = Count of waiting processes!

Mutex vs Binary Semaphore: Crucial Technical Differences

Feature	Mutex (Mutual Exclusion Lock)	Binary Semaphore
Ownership	Has ownership: Jo lock karega, wahi unlock karega.	No ownership: Koi bhi process signal() kar sakta hai.
Purpose	Strictly Locking mechanism (Critical Section).	Signaling mechanism (Coordination & Order).
Priority Inversion	Supports Priority Inheritance Protocol.	Does not support Priority Inheritance.

6.2 Block & Wakeup Implementation (Avoiding Busy Waiting)

```
typedef struct {
    int value;
    struct process *list; // Queue of waiting PCBs
} semaphore;

void wait(semaphore *S) {
    S->value--;
    if (S->value < 0) {
        add_to_queue(S->list, current_process);
        block(); // Process goes to sleep; CPU freed!
    }
}

void signal(semaphore *S) {
    S->value++;
    if (S->value <= 0) {
        process *P = remove_from_queue(S->list);
    }
}
```

```
        wakeup(P); // Wakes up sleeping process to Ready queue
    }
}
```

6.3 Mutex vs Binary Semaphore Comparison

Feature	Mutex	Binary Semaphore
Mechanism	Strict Locking mechanism.	Signaling / Notification mechanism.
Ownership	Locked thread owns it; only owner can unlock.	No ownership; any process can signal.
Priority Inheritance	Supported (prevents priority inversion).	Not supported natively.

7. Producer-Consumer (Bounded Buffer) Problem

Syllabus Coverage: Gateway Classes Slides 65–75 | Circular Buffer, Assembly Race Condition, 3 Semaphores Coordination, Deadlock Pitfall.

7.1 Buffer Architecture & Pointers

Circular array of size `N`. Producer places items at `in`, Consumer extracts items from `out`.

Producer-Consumer (Bounded Buffer) Problem

Circular Ring Buffer & Semaphore Coordination: empty, full, mutex

Shared Circular Buffer (Size N = 8)

X0
[0]

X1
[1]

X2
[2]

—
[3]

—
[4]

—
[5]

—
[6]

—
[7]

Pointers: `in = 3` (Next free slot) | `out = 0` (Next item to consume)

Variables: `count = 3` (Total items currently resident)

Hazard: Simultaneous `count++` & `count--` causes Race Condition!

Semaphores: `empty = 5`, `full = 3`, `mutex = 1`

Correct Semaphore Solutions

PRODUCER PROCESS:

```
wait(empty); // Decrement empty slots
wait(mutex); // Acquire buffer lock
buffer[in] = item; in = (in + 1) % N;
signal(mutex); // Release buffer lock
signal(full); // Increment full items
```

CONSUMER PROCESS:

```
wait(full); // Decrement full items
wait(mutex); // Acquire buffer lock
item = buffer[out]; out = (out + 1) % N;
signal(mutex); // Release buffer lock
signal(empty); // Increment empty slots
```

⚠ THE FAMOUS DEADLOCK TRAP (Exam & Interview Favorite)

Wrong Order: wait(mutex) before wait(empty)

Agar buffer full hai (`empty = 0`):

1. Producer wait(mutex) karke lock le leta hai (`mutex=0`).
2. Producer wait(empty) par BLOCK ho jata hai!
3. Consumer wait(mutex) par block hota hai $\rightarrow$ DEADLOCK!

Golden Rule: Signaling vs Locking Order

Resource availability check (`empty/full`) HAMESHA mutual exclusion lock (`mutex`) se PEHLE aana chahiye!

CORRECT: `wait(empty)` $\rightarrow$ `wait(mutex)`

7.2 Synchronization Semaphores

- `semaphore mutex = 1;` — Ensures mutual exclusion while touching buffer pointers.
- `semaphore empty = N;` — Counts available free slots.
- `semaphore full = 0;` — Counts items available for consumption.

7.3 The Critical Deadlock Trap

Reversing the wait calls to `wait(mutex); wait(empty);` results in an immediate Deadlock when the buffer is full, because the Producer holds the lock while sleeping, starving the Consumer from freeing any space.

8. Classical Problems: Readers-Writers Problem

Syllabus Coverage: Gateway Classes Slides 76–92 | Conflict Matrix, First Readers-Writers Problem, Execution Trace, Writer Starvation.

8.1 Access Rules & Protocols

Readers may read concurrently; Writers require exclusive mutual exclusion.

- `semaphore mutex = 1;` — Protects reader counter `rc`.
- `semaphore db = 1;` — Mutual exclusion lock for the database.

Readers-Writers Problem Architecture

Concurrent Readers, Exclusive Writer & Writer Starvation Mechanics

Conflict & Permission Matrix

Reader & Reader: ✓ ALLOWED (Concurrent)

Reader & Writer: ✗ FORBIDDEN (Conflict)

Writer & Writer: ✗ FORBIDDEN (Conflict)

Key Insight: Only Writers require strict mutual exclusion!

The Semaphore Protocols

READER PROTOCOL (`rc` = reader count):

```
wait(mutex); rc++;
if (rc == 1) wait(db); // 1st reader locks DB for all readers
signal(mutex);
/* READ DATABASE CONCURRENTLY */
wait(mutex); rc--;
if (rc == 0) signal(db); // Last reader unlocks DB
signal(mutex);
```

WRITER PROTOCOL:

```
wait(db); // Exclusively locks DB
/* WRITE DATABASE */
signal(db); // Releases DB
```

⚠ THE PROBLEM OF WRITER STARVATION (First Readers-Writers Problem)

Agar Readers continuously aate rahein (R1 enters, then R2 enters before R1 finishes, then R3 enters...),
toh reader count 'rc' KABHI BHI 0 NAHI HOGA!
 Nateeja: Database lock 'db' kabhi release nahi hoga, aur waiting Writer process hamesha ke liye starvation
 mein phas jayega. Isse solve karne ke liye Fair / Writer-Priority Solution use kiya jata hai.

8.2 The "First In, Last Out" Semaphore Lock Rule

```
// First reader locks DB for all readers
if (rc == 1) wait(db);

// Last reader unlocks DB for writers
if (rc == 0) signal(db);
```

8.3 Starvation Dilemma

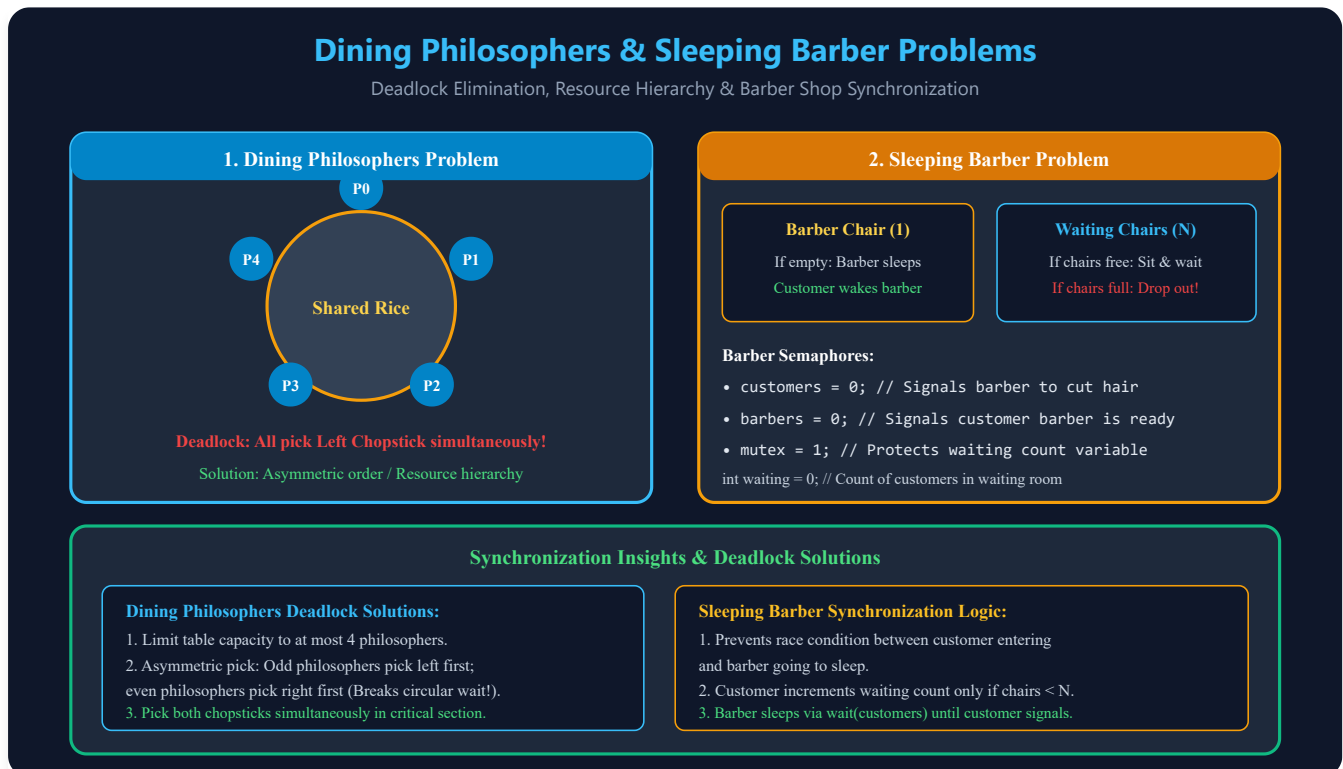
If new readers arrive continuously before active readers finish, `rc` never reaches zero, leading to indefinite starvation of pending writers.

9. Classical Problems: Dining Philosophers & Sleeping Barber

Syllabus Coverage: Gateway Classes Slides 93–107 & AKTU Syllabus | Circular Wait Deadlock, Asymmetric Solution, Barber Synchronization.

9.1 Dining Philosophers Deadlock Prevention

Deadlock arises when all philosophers simultaneously grab their left chopstick. Solved using asymmetric pickup or limiting diners to [N-1](#).



9.2 Sleeping Barber Mechanics

- customers = 0 : Barber blocks on wait(customers) until customer wakes him.
- barbers = 0 : Customer blocks on wait(barbers) until barber chair is vacant.
- mutex = 1 : Protects waiting counter from concurrent updates.

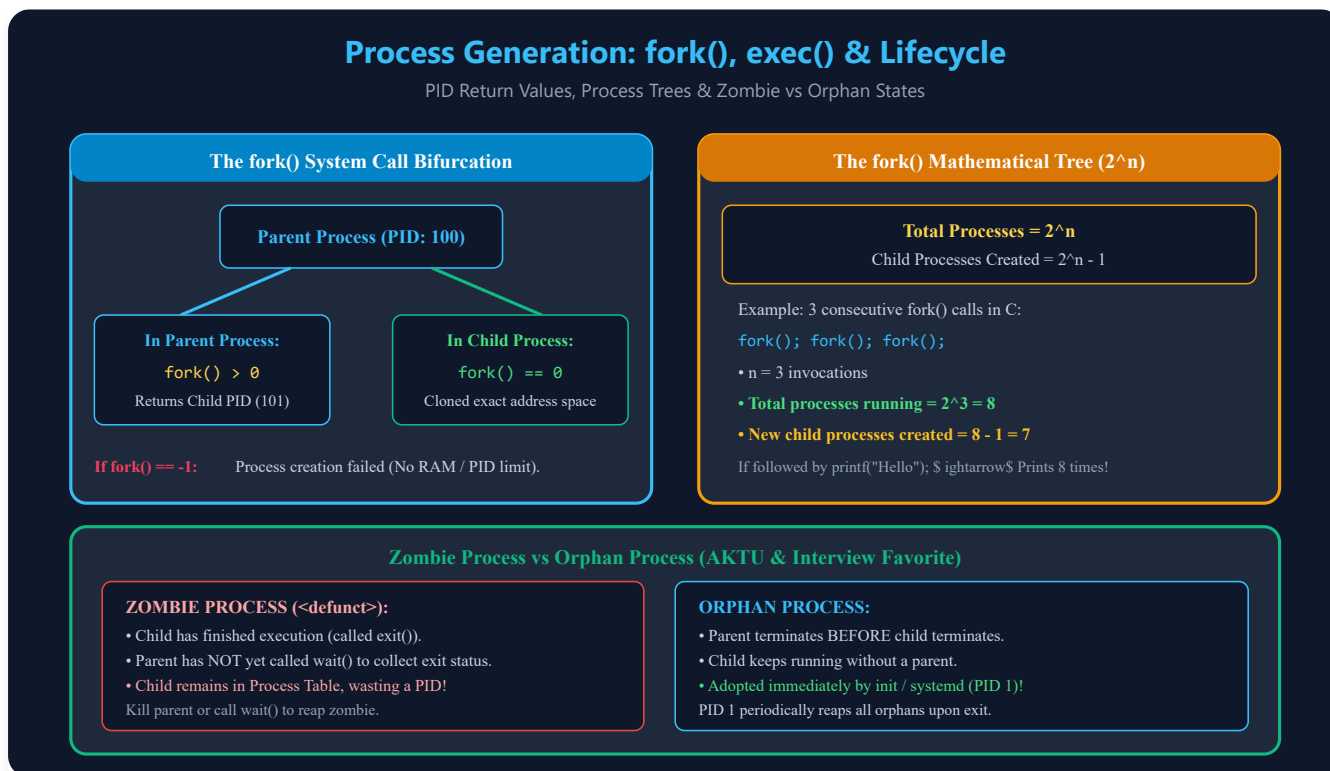
10. Process Generation, Forking & Lifecycle

Syllabus Coverage: Gateway Classes Slides 121–128 | Forking vs Spawning, Return Values, 2^n Tree Mathematics, Zombie vs Orphan Processes.

10.1 The fork() System Call

Clones the calling parent process into an identical child address space.

- `pid == 0` in Child process.
- `pid > 0` in Parent process (value is child PID).
- `pid == -1` on process creation failure.



10.2 Mathematical Process Tree Growth

Formula for n consecutive `fork()` statements:

Total Processes = 2^n
Total Child Processes Created = $2^n - 1$

10.3 Zombie vs Orphan Process Comparison

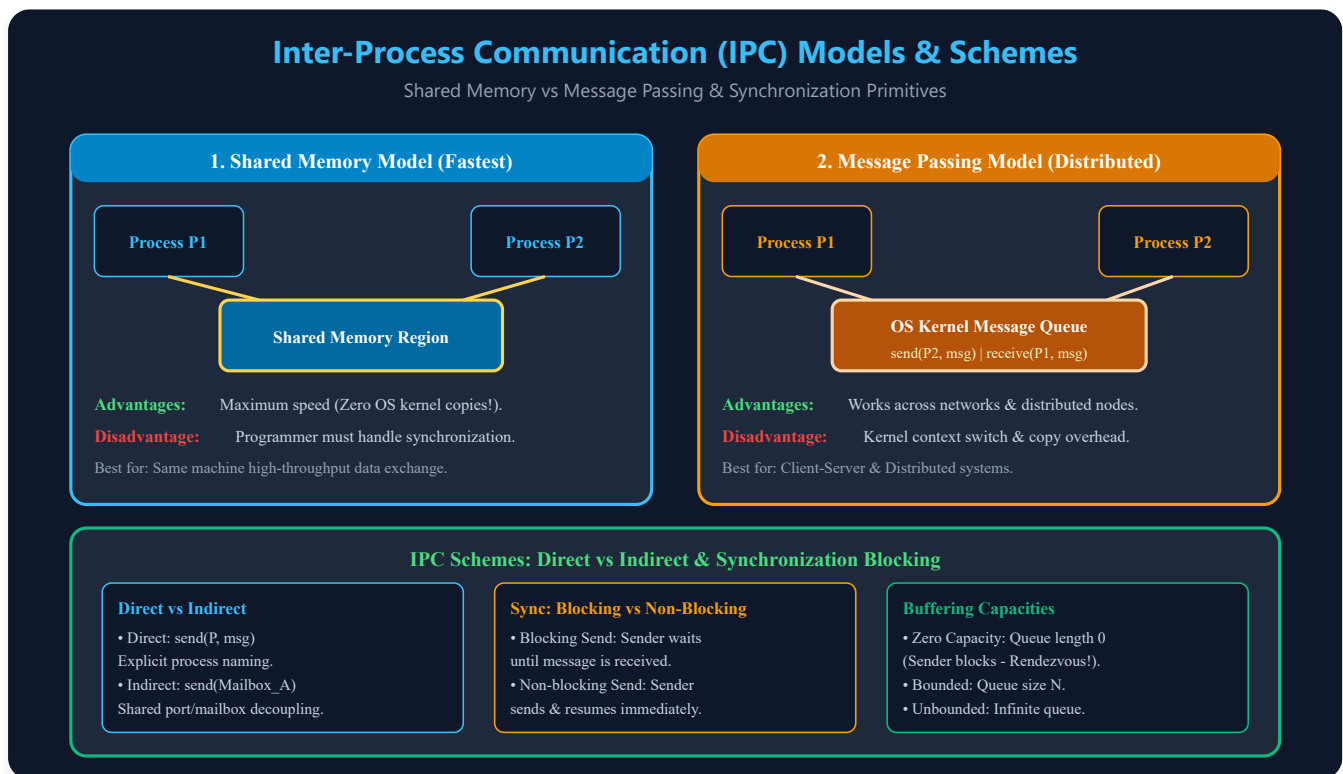
Feature	Zombie Process (<defunct>)	Orphan Process
Terminated First	Child terminated first; Parent still alive.	Parent terminated first; Child still running.
Process Table Entry	Entry remains in Process Table (wastes PID).	Active normal process entry.
Resolution / Adoption	Parent must call <code>wait()</code> to reap status.	Immediately adopted by <code>init</code> / <code>systemd</code> (PID 1).

11. Inter-Process Communication (IPC) Models & Schemes

Syllabus Coverage: Gateway Classes Slides 129–137 | Shared Memory vs Message Passing, Direct vs Indirect, Blocking Semantics, Buffering Schemes.

11.1 The Two Fundamental Models

- **Shared Memory:** Fastest mechanism; zero kernel overhead after setup; requires programmer-level semaphore synchronization.
- **Message Passing:** Kernel coordinates message queues; slower due to context switching; ideal for distributed systems.

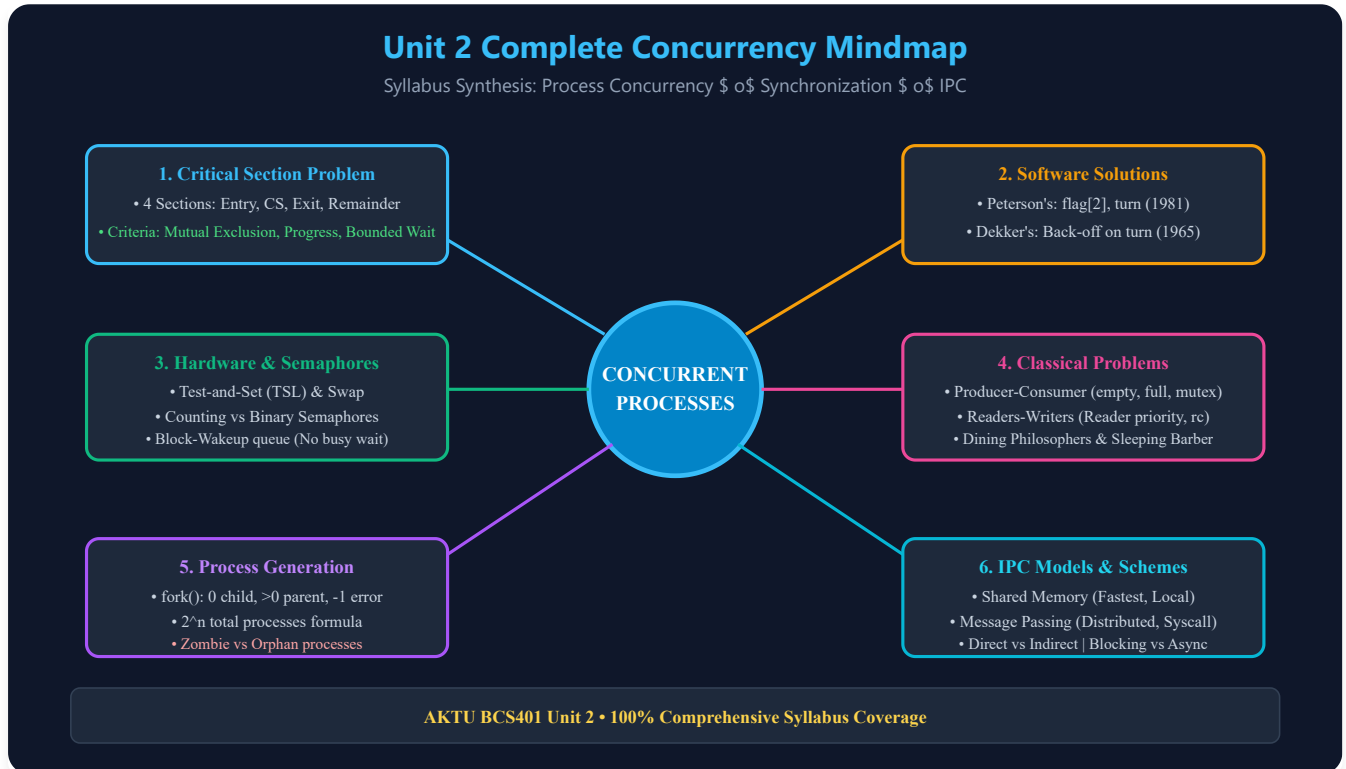


11.2 Architectural Comparison

Feature	Shared Memory	Message Passing
Communication Path	User memory address space.	Kernel message buffers.
Speed	Extremely Fast (RAM bandwidth).	Slower (Syscalls + data copying).
Synchronization	Manual (Semaphores / Mutexes).	Automatic by kernel.
Network Scalability	Single machine only.	Works across network clusters.

12. Unit 2: AKTU PYQs & Interview Cheatsheet

Syllabus Coverage: Gateway Classes Slides 40, 60, 92, 106, 120, 136 | Solved AKTU Question Bank (2014–2023), Top Interview Traps, Concurrency Mindmap.



12.1 High-Yield Solved Exam Questions

- **Critical Section Criteria:** Mutual Exclusion (Primary), Progress (Primary), Bounded Waiting (Secondary), Speed Neutrality.
- **Peterson's Algorithm:** Proves mutual exclusion via single integer `turn`. Bounded wait $k=1$.
- **Overcoming Busy Waiting:** Use `block()` and `wakeup()` queue-based semaphore implementation.
- **Producer-Consumer Trap:** Always place counting semaphore (`empty/full`) before mutex to prevent deadlock.
- **Process Tree Formula:** n consecutive forks yield 2^n total processes and $2^n - 1$ child processes.
- **Zombie vs Orphan:** Zombie = Dead child waiting for parent's `wait()`; Orphan = Running child adopted by PID 1.